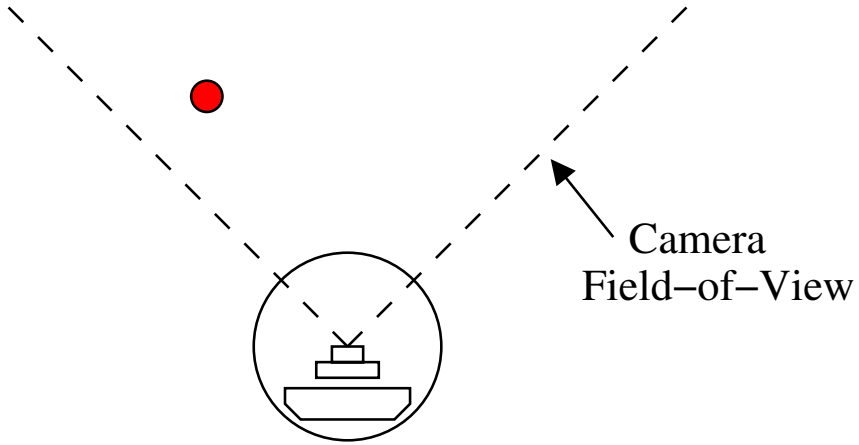
PID Control

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Tracking an Object

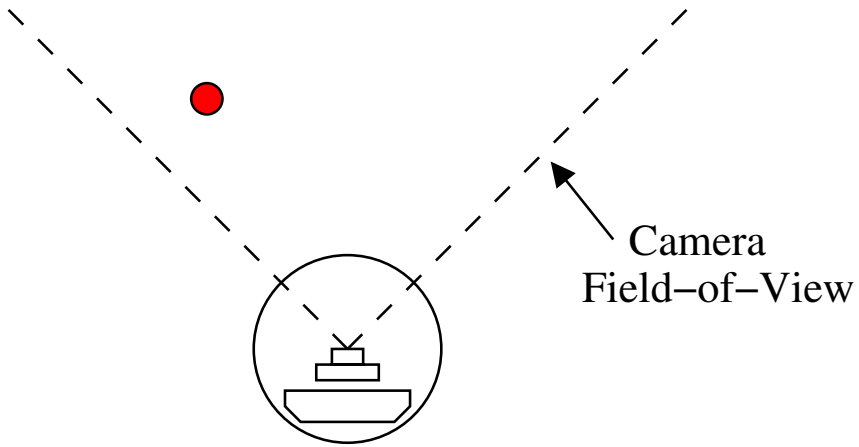
- Imagine that there is an object somewhere in front of our robot



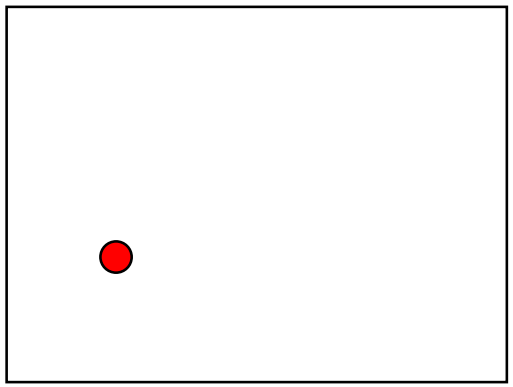
- How to turn the robot such that it points directly to the object?

Tracking an Object

- Again, top view



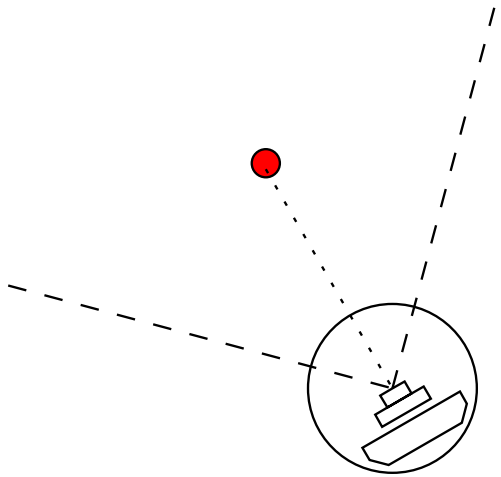
- Here is from the camera point-of-view



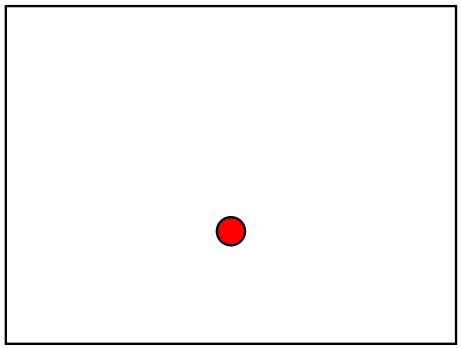
- The object detection node may say it sees an object at $(150, 160)$

Tracking an Object

- The goal is to turn the robot such that it points directly to the object the object



- Here is from the camera point-of-view



- The object detection node may say it sees an object at $(320, 160)$

Tracking an Object

- The simplified goal is to make the object appear at the center of the camera (x-coordinate is at 320)
- Execution (same as one of our lab)
  - If x-coordinate of the object is in between 315 to 325, stop moving
    - `linear.x = 0.0` and `angular.z = 0.0`
  - If x-coordinate of the object is less than 315 (on the left side), tell the robot to make a stationary turn left at speed 0.5
    - `linear.x = 0.0` and `angular.z = 0.5`
  - If x-coordinate of the object is greater than 325 (on the right side), tell the robot to make a stationary turn right at speed 0.5
    - `linear.x = 0.0` and `angular.z = -0.5`

Tracking an Object

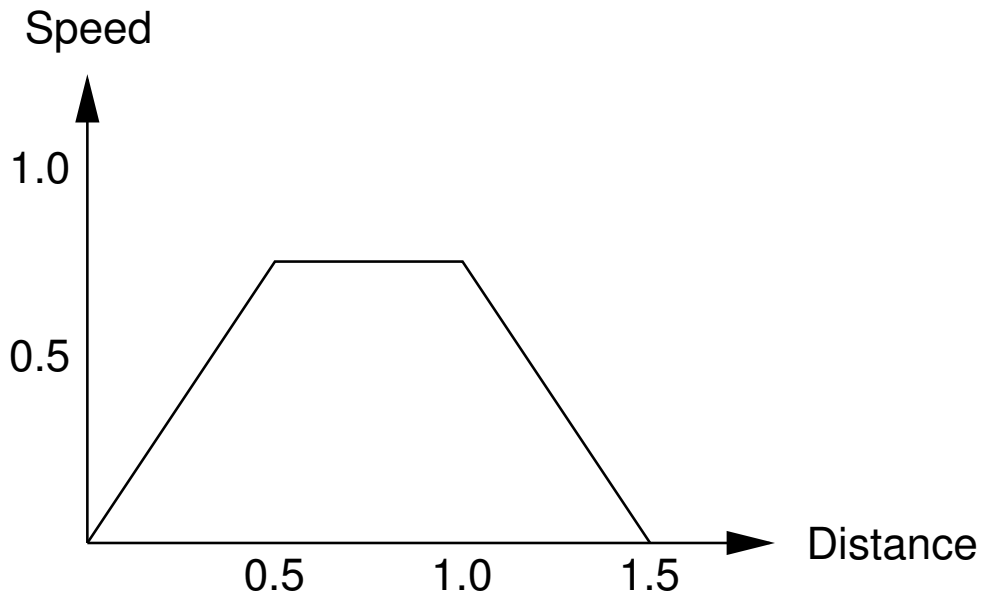
- This simplified method is suitable for tracking a stationary object
  - An object that does not move or move very slowly
- If an object moves around the robot in circle:
  - At speed greater than 0.5 radian per second
    - Eventually the robot will loose track of the object because the object moves out of the camera view
  - At speed exactly 0.5 radian per second
    - The robot will not be able to point itself directly at the object
    - The robot will not loose track of the object but will not be able to catch up with it
  - At speed less than 0.5 radian per second
    - The robot will be able to catch up with the object eventually
    - The robot will stutter (not smooth) especially when the object is at the center a move slightly out of center

Control System

- A control system is a device, or set of devices, that manages, commands, directs or regulates the behavior of other devices or systems
- In robot application, the control system is the software that manage/regulate commands that control the behavior of robots
  - Tells a robot to move faster or slower
  - Tells a robot to turn left or right
- Type of Control Systems:
  - **Open Loop**: Output is generated purely based on the current input
  - **Closed Loop**: Current output is taken into consideration and corrections are made based on feedback

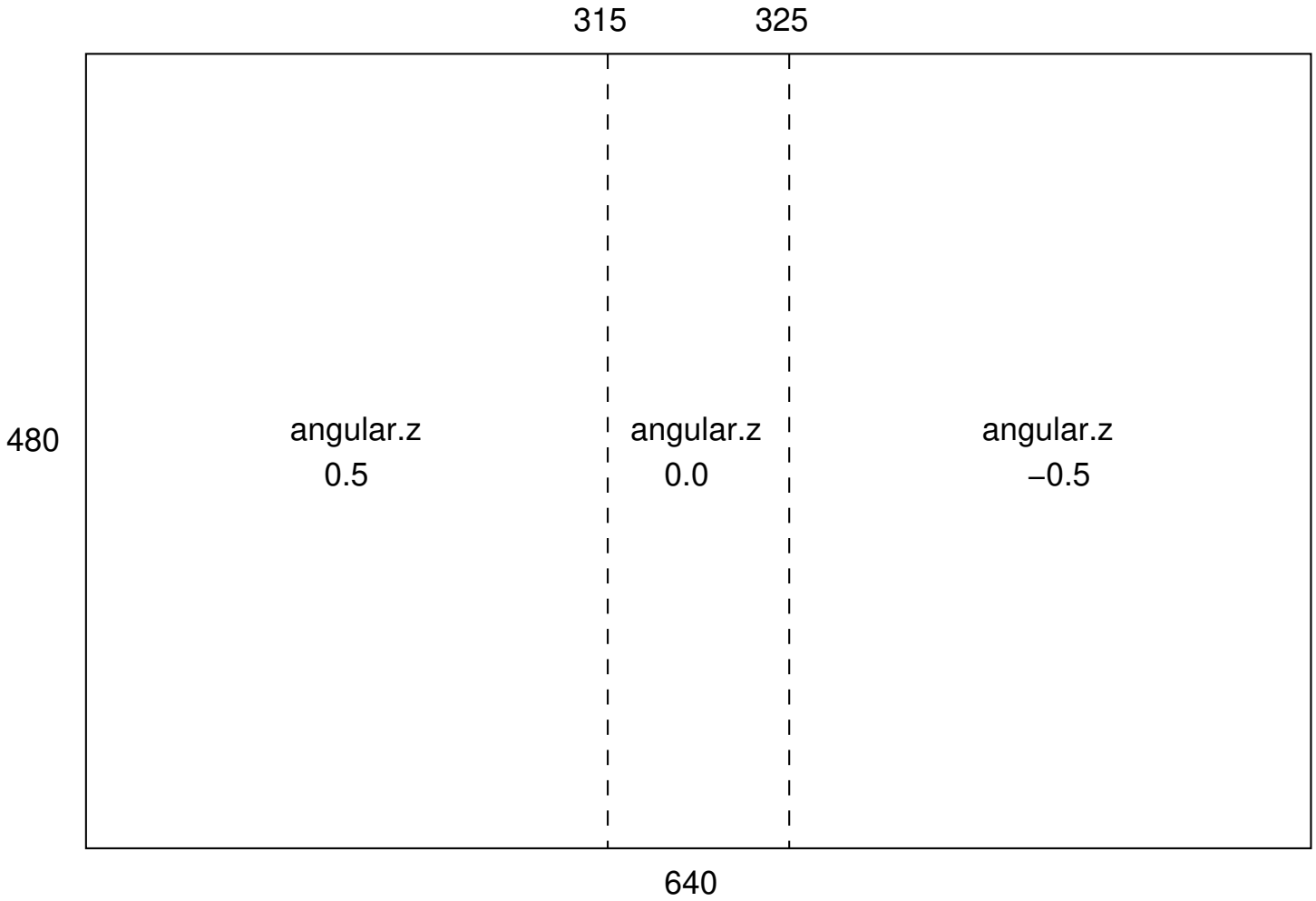
Open Loop Examples

- **Project 1 (Sennott Rover)**: We adjust the command `linear.x` and `angular.z` based only on the odometry data.



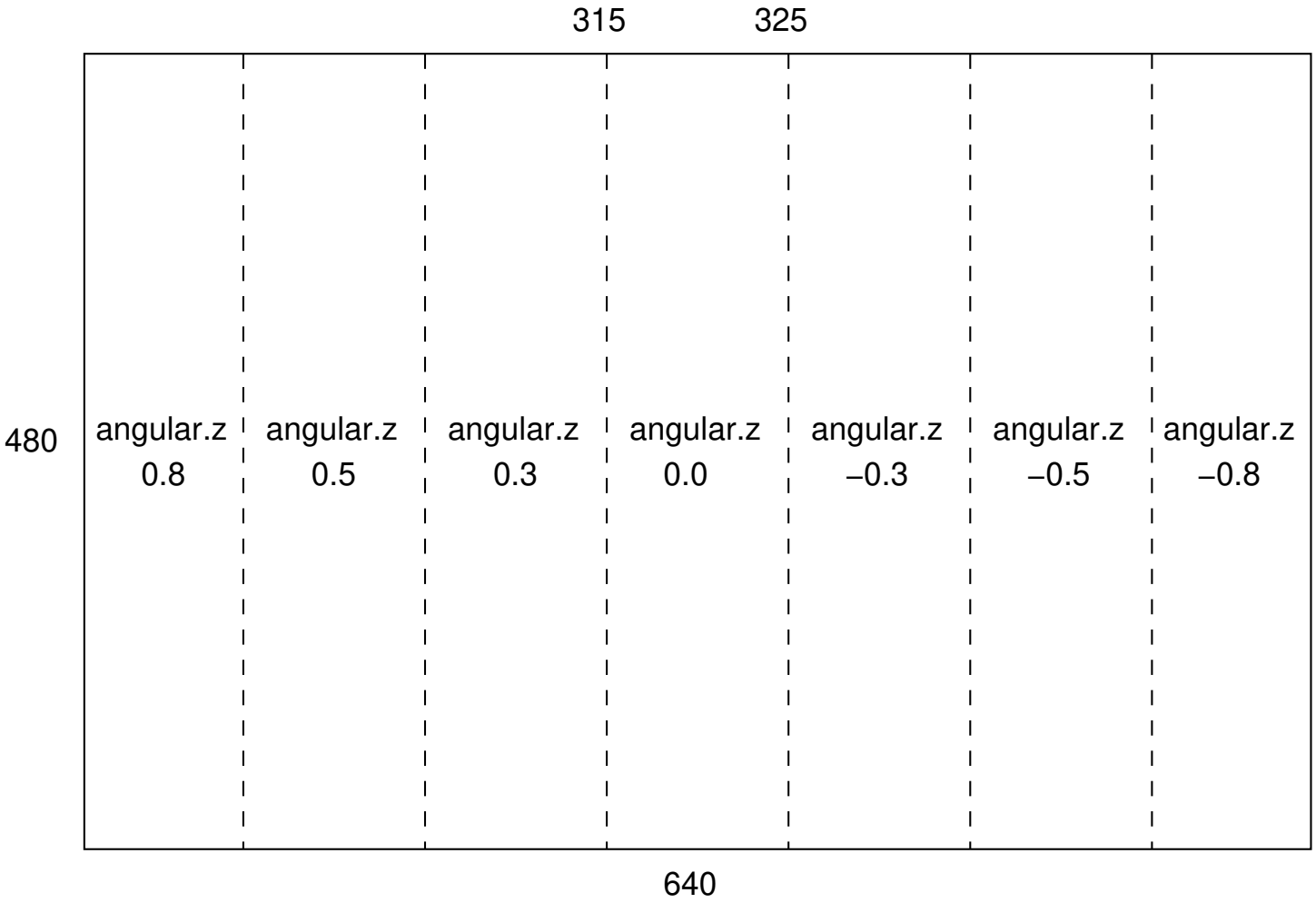
Open Loop Examples

- **Lab (Blob Tracker)**: We adjust the command `angular.z` based only on the pointer location and the odometry data



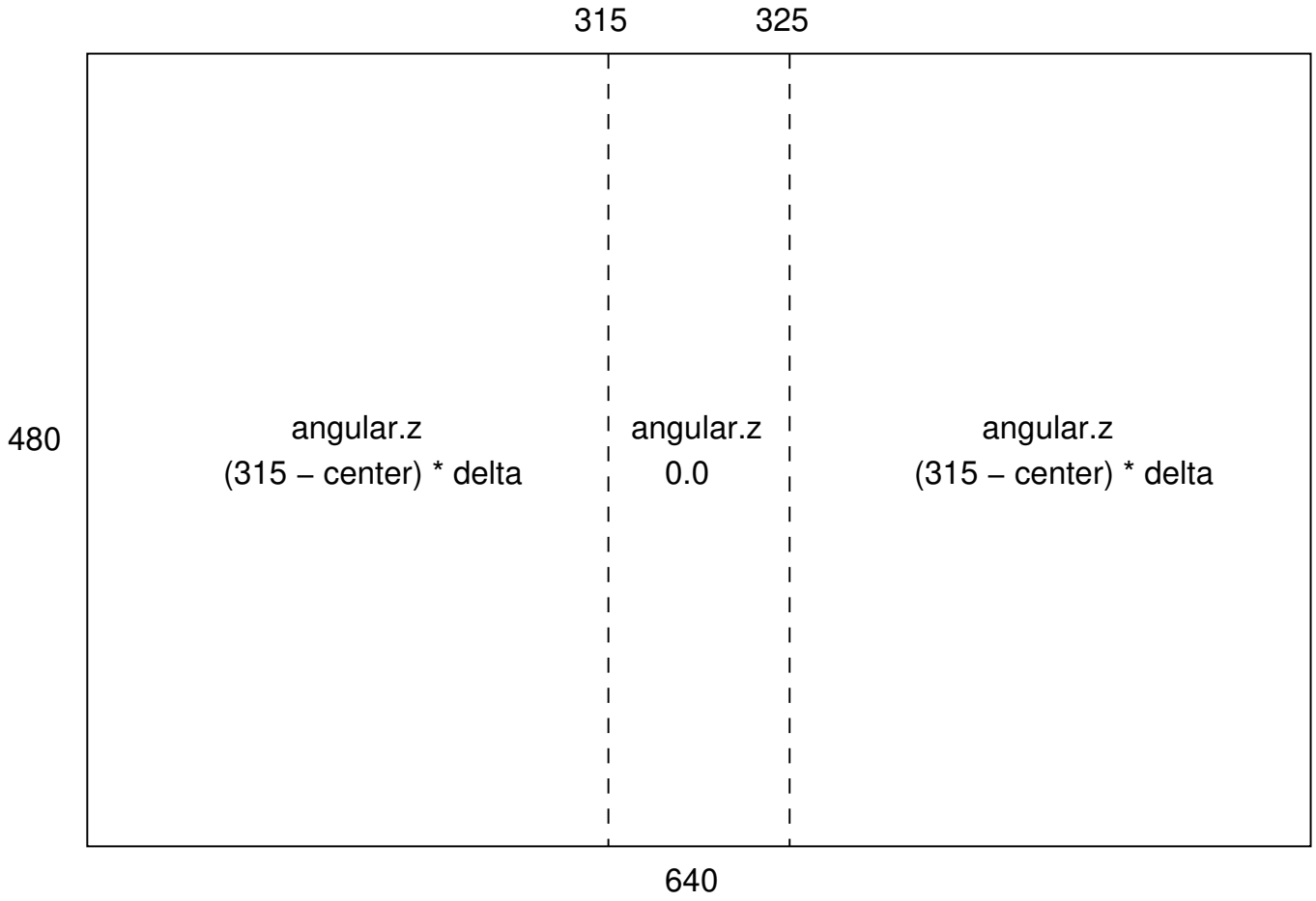
Open Loop Examples

- **Lab (Blob Tracker)**: For better follower, you may have various speed based on the pointer location and the odometry data



Open Loop Examples

- **Lab (Mouse Tracker)**: For even better result, the turn speed should be proportional to the distance from center



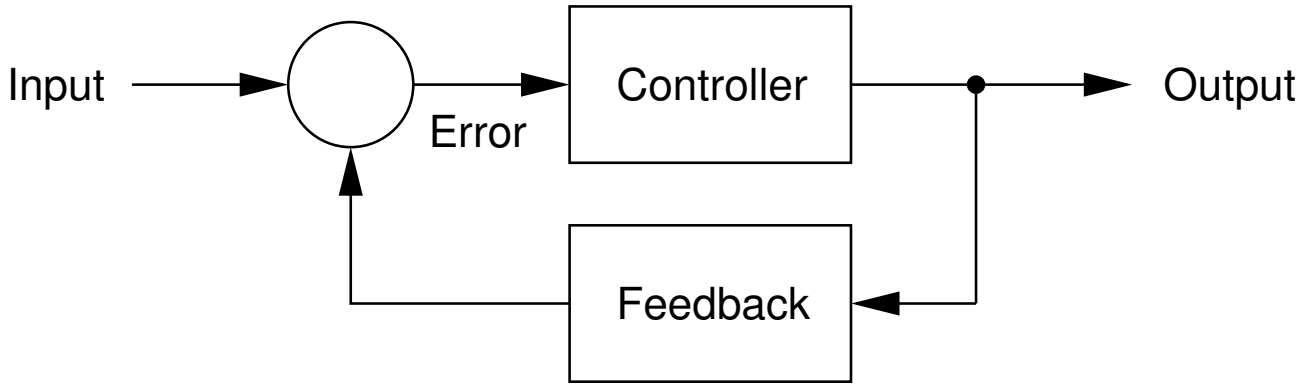
  - **center** is the x-coordinate of the object
  - delta ≥ 0

Problems with Open Loop Control

- Problem with object tracking and line following application
- Oscillation:
  - Hard to see when the robot moves at slow speed
  - At faster speed, the robot may oscillate trying to keep at center
- Lost track:
  - The robot may not turn fast enough
- Overcompensate:
  - Over shoot the target (cause oscillations)

Closed Loop Control

- Closed Loop Control



- The goal is to generate output signal to minimize the error
- Example: In object tracking application
  - Input the location of an object (according to the image)
  - The error is the number of pixels of the object from the center of the screen
  - The output is the command (`Twist`)

PID Control

- Proportional-Integral-Derivative (PID) Controller is a closed loop control
- Let
  - $z(t)$ be the control value at time t (will be turned in to `angular.z`),
  - $e(t)$ be the error at time t (e.g., number of pixel of the center of the object to the center of the screen), and
  - K_p, K_i, K_d are non-negative coefficients

$$z(t) = K_p e(t) + K_i \int_0^t e(t)dt + K_d \frac{de}{dt}$$

- The meaning of $z(t)$, $e(t)$, $e(t)dt$, and $\frac{de}{dt}$ depend on application

PID Control

- PID:

$$z(t) = K_p e(t) + K_i \int_0^t e(t)dt + K_d \frac{de}{dt}$$

- From the above equation $z(t)$ is adjusted based on the following:
  1. **Proportional**: magnitude of the error $(e(t))$
  2. **Integral**: accumulated magnitude of the error $(\int_0^t e(t)dt)$, and
  3. **Differential**: speed of error $(\frac{de}{dt})$
- Specify values of K_p, K_i, and K_d are difficult
  - These values depend on type of application and characteristic of the robot

Proportional

- Proportional portion of PID

$$K_p e(t)$$

- This term allows you to adjust the speed based on the magnitude of the error
- The larger the error, the faster the robot should turn
- The sign of the error effects the sign of the output (direction)

Differential

- Differential portion of PID

$$K_d \frac{de}{dt}$$

- $\frac{de}{dt}$ is the rate of the error
  - This can be as simple as $e(t) - e(t-1)$
- If the robot turn fast enough, the sign of the rate of the error will be different than the sign of the magnitude of the error $e(t)$
  - To prevent over compensate, we should slow the speed down a little bit
- This also help when the robot turns too slow, the sign of the error rate will be the same as the magnitude of the error which makes the robot turn even faster

PID Control

- Sometime P and D portions are not enough
- Moving object:
  - The P part makes the robot turn at the exact speed of the object movement
  - The D part will be 0
  - Never be able to catch up with it
- Balancing application (Segway)
  - The P and D parts can make it stands on its two wheels
  - But it may slowly drift forward or backward

Integral

- Integral portion of PID

$$K_i \int_0^t e(t)dt$$

- This is the accumulating of error over time
- This helps catching up with moving object
- This also helps with drifting

Find the values of K_p, K_i, and K_d

- PID Control

$$z(t) = K_p e(t) + K_i \int_0^t e(t)dt + K_d \frac{de}{dt}$$

- The values of K_p, K_i, and K_d generally based on robot application
  - A way to find one these values is simply test it
- First, find K_p by setting K_i and K_d to 0s
  - Slowly adjust K_p until the robot start oscillating
- Second, fix K_p from the first step, setting K_i to 0
  - Slowly adjust K_d until the robot stop oscillating
- Then, adjusting K_i.